

1. Контейнеры

В Bootstrap всё начинается с контейнера. Это базовый строительный блок, который содержит, выравнивает и центрирует содержимое страницы. Контейнеры бывают трех типов: фиксированный (.container), на всю ширину (.container-fluid) и адаптивные контейнеры (.container-sm, .container-md и т.д.).

```
<div class="container">
  <!-- Содержимое с отступами по бокам -->
</div>

<div class="container-fluid">
  <!-- Контейнер на всю ширину экрана -->
</div>

<div class="container-lg">
  <!-- Адаптивный контейнер (будет на всю ширину только на маленьких экранах) -->
</div>
```

2. Сетка

Bootstrap использует систему «12 колонок», 5 адаптивных ярусов, препроцессоров Sass и предустановленных классов.

Очень страшные слова, поэтому разберем на небольшом примере:

Представь, что ширина экрана разделена на 12 равных частей.

```
<div class="row">
  <div class="col-4">Заняли 4 колонки</div>
  <div class="col-8">Заняли 8 колонок</div>
  <!-- Итого заняли 12 колонок -->
</div>
```

Почему именно 12? Число 12 удобно тем, что делится на много частей: на 2, 3, 4, 6. Это дает гибкость в верстке. Например, можно создать макет из двух колонок по 6, трех колонок по 4, четырех колонок по 3 или шести колонок по 2.

Адаптивные ярусы (брейкпоинты)

Адаптивные ярусы (брейкпоинты) — это точки перелома, при которых макет страницы меняется в зависимости от ширины экрана. Bootstrap имеет 6 предопределенных брейкпоинтов, основанных на размерах популярных устройств.

Ярус	Класс	Когда срабатывает	Какие устройства
Очень маленький	xs	<576px	Старые телефоны
Маленький	sm	≥576px	Современные телефоны
Средний	md	≥768px	Планшеты
Большой	lg	≥992px	Ноутбуки
Очень большой	xl	≥1200px	Большие мониторы

3. Колонки

Поскольку сетка Bootstrap построена на flexbox, мы можем использовать различные утилиты для выравнивания колонок по вертикали и горизонтали.

Верх (align-items-start)

Колонка 2

Колонка 3

Центр (align-items-center)

Колонка 2

Колонка 3

Выравнивание по нижнему краю (align-items-end)

Колонка 2

Колонка 3

Вертикальное выравнивание для всех колонок в ряду

Классы `.align-items-*` на родителе `.row` выравнивают все колонки одновременно:

```
<div class="row align-items-start">
  <div class="col">Колонка 1</div>
  <div class="col">Колонка 2</div>
  <div class="col">Колонка 3</div>
</div>
```

Также можно выровнять каждую колонку индивидуально с помощью классов `.align-self-*`:

`.align-self-start`

`.align-self-center`

`.align-self-end`

```
<div class="row">
  <div class="col align-self-start">...</div>
  <div class="col align-self-center">...</div>
  <div class="col align-self-end">...</div>
</div>
```

Горизонтальное выравнивание

Классы `.justify-content-*` на ряду управляют расположением колонок по горизонтали:

`.justify-content-start`

Колонка 2

`.justify-content-center`

Колонка 2

`.justify-content-end`

Колонка 2

`.justify-content-around`

Колонка 2

`.justify-content-between`

Колонка 2

```
<div class="row justify-content-start">
  <div class="col-4">...</div>
  <div class="col-4">...</div>
</div>
```

Обертывание колонок

Если сумма колонок в ряду превышает 12, лишние колонки автоматически переносятся на новую строку:

`.col-9`

`.col-4`
Поскольку $9 + 4 = 13 > 12$, этот div шириной в 4 колонки переносится на новую строку как один непрерывный блок.

`.col-6`
Последующие колонки продолжаютс вдоль новой строки.

```
<div class="row">
  <div class="col-9">.col-9</div>
  <div class="col-4">.col-4 (переносится)</div>
  <div class="col-6">.col-6</div>
</div>
```

Разрывы колонок

Пустой `<div class="w-100"></div>` создаёт принудительный перенос на новую строку в нужном месте:

.col-6 .col-sm-4	.col-6 .col-sm-4
.col-6 .col-sm-4	.col-6 .col-sm-4

```
<div class="row">
  <div class="col-6 col-sm-4">1</div>
  <div class="col-6 col-sm-4">2</div>
</div>
<div class="w-100"></div>
<div class="col-6 col-sm-4">3</div>
<div class="col-6 col-sm-4">4</div>
</div>
```

Комбинация с утилитами отображения позволяет делать разрыв только на определённых устройствах:

```
<!-- Разрыв только на md и выше -->
<div class="w-100 d-none d-md-block"></div>
```

Изменение порядка колонок

Классы **.order-*** меняют визуальный порядок колонок независимо от их порядка в HTML (доступны значения от 1 до 5):

Второй в HTML, но первый по порядку (.order-1)	Третий в HTML, но второй по порядку (.order-2)	Первый в HTML, но третий по порядку (.order-3)
--	--	--

```
<div class="row">
  <div class="col order-3">Первый в HTML, но третий</div>
  <div class="col order-1">Второй в HTML, но первый</div>
  <div class="col order-2">Третий в HTML, но второй</div>
</div>
```

Классы **.order-first** и **.order-last** помещают колонку соответственно в начало или конец:

Третий в HTML, но первый (.order-first)	Второй в HTML, без порядка	Первый в HTML, но последний (.order-last)
---	----------------------------	---

```
<div class="row">
  <div class="col order-last">Первый в HTML, но последний</div>
  <div class="col">Второй в HTML</div>
  <div class="col order-first">Третий в HTML, но первый</div>
</div>
```

Классы смещения .offset-*

Классы **.offset-*** увеличивают левый отступ колонки на указанное количество колонок:

.col-md-4	.col-md-4 .offset-md-4
.col-md-3 .offset-md-3	.col-md-3 .offset-md-3
.col-md-6 .offset-md-3	

```
<div class="row">
  <div class="col-md-4">.col-md-4</div>
  <div class="col-md-4 offset-md-4">.col-md-4 .offset-md-4</div>
</div>
```

Утилиты полей (margin)

Утилиты **.ms-auto** (margin-left: auto) и **.me-auto** (margin-right: auto) позволяют прижимать колонки к краям или создавать переменные отступы:

.col-md-4

.col-md-4 .ms-auto

.col-md-3 .ms-md-auto

.col-md-3 .ms-md-auto

.col-auto .me-auto

.col-auto

```
<div class="row">
  <div class="col-md-4">.col-md-4</div>
  <div class="col-md-4 ms-auto">.col-md-4 .ms-auto</div>
</div>
<div class="row">
  <div class="col-auto me-auto">.col-auto .me-auto</div>
  <div class="col-auto">.col-auto</div>
</div>
```

Автономные классы колонок

Классы **.col-*** можно использовать вне **.row** для задания ширины любому элементу — в таком случае отступы (gutter) не применяются:

.col-3: ширина 25%

.col-sm-9: ширина 75% на sm и выше

<div class="col-3 p-3">.col-3: ширина 25%</div>
<div class="col-sm-9 p-3">.col-sm-9: ширина 75% на sm и выше</div>

Это удобно для создания плавающих элементов, например изображений с обтеканием текстом:

Абзац текста, который обтекает изображение. Класс `.clearfix` нужен для правильной очистки обтекания. Изображение имеет классы `.col-md-6` (ширина 50% на md и выше), `.float-md-end` (обтекание справа) и отступы для красоты.

Такой подход позволяет создавать адаптивные макеты с обтеканием изображений текстом.

Изображение

```
<div class="clearfix">
  
  <p>Текст, обтекающий изображение...</p>
</div>
```

4. Отступы

Отступы между колонками используются для гибкого размещения и выравнивания содержимого в системе сеток Bootstrap. Они создаются с помощью `padding` у колонок и отрицательных `margin` у рядов.

Как это работает

- Горизонтальные отступы между столбцами создаются с помощью `padding`. Мы устанавливаем `padding-right` и `padding-left` для каждого столбца и используем отрицательное значение `margin` для смещения в начале и конце каждой строки, чтобы выровнять содержимое.
- По умолчанию отступ имеет ширину `1.5rem` (`24px`). Это позволяет сопоставить сетку с масштабом отступов и интервалов полей.
- Отступы могут быть быстро отрегулированы с помощью специальных классов.

Горизонтальные отступы (.gx-*)

Классы **.gx-*** управляют шириной горизонтальных промежутков между колонками. Доступны значения от 0 до 5:

Колонка с отступом

Колонка с отступом

<div class="container px-4 text-center">
 <div class="row gx-5">
 <div class="col">
 <div class="p-3">Содержание столбца</div>
 </div>
 <div class="col">
 <div class="p-3">Содержание столбца</div>
 </div>
 </div>
</div>

`.gx-5` создает большие отступы между колонками. При использовании больших отступов может потребоваться корректировка контейнера с помощью утилит `padding`, например `.px-4`.

Вертикальные отступы (.gy-*)

Классы `.gy-*` управляют вертикальными промежутками между строками колонок:

Колонка 1

Колонка 2

Колонка 3

Колонка 4

```
<div class="container text-center">
  <div class="row gy-5">
    <div class="col-6">Колонка 1</div>
    <div class="col-6">Колонка 2</div>
    <div class="col-6">Колонка 3</div>
    <div class="col-6">Колонка 4</div>
  </div>
</div>
```

Горизонтальные и вертикальные отступы (.g-*)

Классы `.g-*` одновременно задают и горизонтальные, и вертикальные отступы:

Колонка с отступами со всех сторон

Колонка с отступами со всех сторон

Колонка с отступами со всех сторон

Колонка с отступами со всех сторон

```
<div class="container text-center">
  <div class="row g-4">
    <div class="col-6">Колонка</div>
    <div class="col-6">Колонка</div>
    <div class="col-6">Колонка</div>
    <div class="col-6">Колонка</div>
  </div>
</div>
```

Без промежутков между колонками (.g-0)

Класс `.g-0` полностью удаляет отступы между колонками. Он убирает отрицательные `margin` у `.row` и горизонтальные `padding` у всех дочерних колонок:

`.col-sm-6 .col-md-8`

`.col-6 .col-md-4`

```
<div class="row g-0 text-center">
  <div class="col-sm-6 col-md-8">.col-sm-6 .col-md-8</div>
  <div class="col-6 col-md-4">.col-6 .col-md-4</div>
</div>
```

Это полезно для создания "сквозных" дизайнов, где элементы должны прилегать друг к другу вплотную.

Ряды столбцов с отступами

Классы отступов можно комбинировать с классами `.row-cols-*` для создания адаптивных сеток с управляемыми промежутками:

Колонка ряда

Колонка ряда

Колонка ряда

Колонка ряда

Колонка ряда

Колонка ряда

Колонка ряда

Колонка ряда

```
<div class="row row-cols-2 row-cols-lg-4 g-4">
  <div class="col">
    <div class="p-3">Колонка ряда</div>
  </div>
  <div class="col">
    <div class="p-3">Колонка ряда</div>
  </div>
  ...
</div>
```

Доступные значения отступов

Классы отступов строятся из карты Sass `$gutters`, которая унаследована от карты `$spacers`. Доступны значения от 0 до 5:

Класс	Значение	Размер
.g-0, .gx-0, .gy-0	0	0
.g-1, .gx-1, .gy-1	\$spacer * 0.25	0.25rem (4px)
.g-2, .gx-2, .gy-2	\$spacer * 0.5	0.5rem (8px)
.g-3, .gx-3, .gy-3	\$spacer	1rem (16px)
.g-4, .gx-4, .gy-4	\$spacer * 1.5	1.5rem (24px) - значение по умолчанию
.g-5, .gx-5, .gy-5	\$spacer * 3	3rem (48px)

Классы можно делать адаптивными, добавляя брейкпоинты: `.gx-md-4`, `.gy-lg-2`, `.g-sm-3` и т.д.

Автоматическая компоновка колонок

Равная ширина

Если не указывать цифру в классе col, все колонки будут одинаковой ширины:

Колонка 1

Колонка 2

Колонка 3

```
<div class="row">
  <div class="col">Колонка 1</div>
  <div class="col">Колонка 2</div>
  <div class="col">Колонка 3</div>
</div>
```

Установка ширины одной колонки

Можно задать ширину одной колонки, остальные подстроятся автоматически:

1

2(шире, col-6)

3

```
<div class="row">
  <div class="col">1</div>
  <div class="col-6">2</div>
  <div class="col">3</div>
</div>
```

Переменная ширина контента (col-auto)

Класс col-auto заставляет колонку занимать ровно столько места, сколько нужно её содержимому:

1

Переменная ширина контента

3

```
<div class="row">
  <div class="col col-lg-2">1</div>
  <div class="col-md-auto">Переменная ширина контента</div>
  <div class="col col-lg-2">3</div>
</div>
```

Пример использования адаптивности

Рассмотрим пример с маркетплейсом Yargy Market, где нужно разместить карточки товаров. Для каждого типа устройства требуется своя разметка, так как на экране телефона помещается меньше информации, чем на большом мониторе. Используя классы Bootstrap, мы можем задать разное количество колонок для разных устройств.

Товар №1

Товар №2

Товар №3

Товар №4

Визуализация расположения на разных устройствах:

Телефон

Товар №1

Товар №2

Товар №3

Товар №4

1 колонка (col-12)

Планшет

Товар №1

Товар №2

Товар №3

Товар №4

2 колонки (col-md-6)

Ноутбук

Товар №1

Товар №2

Товар №3

Товар №4

3 колонки (col-lg-4)

Монитор

Товар№№1

Товар№№2

Товар№№3

Товар№№4

4 колонки (col-xl-3)

Классы .row-cols-*

Классы `.row-cols-*` — это специальные классы Bootstrap, которые задают количество колонок в ряду прямо на родительском контейнере `.row`. Вместо того чтобы писать класс для каждой колонки отдельно, мы пишем один класс на весь ряд. Это особенно удобно при работе с динамическим контентом, когда количество элементов может меняться.

Синтаксис:

- `.row-cols-{количество}` — одинаковое количество колонок на всех экранах
- `.row-cols-{breakpoint}-{количество}` — адаптивное количество колонок
- `.row-cols-auto` — колонки подстраиваются под ширину содержимого

Пример с разным количеством на разных устройствах:

1

2

3

4

5

6

7

8

```
<div class="row row-cols-1 row-cols-sm-2 row-cols-md-3 row-cols-lg-6">  
  <div class="col">1</div>  
  <div class="col">2</div>  
  ...  
</div>
```

Пример с `.row-cols-auto`:

Короткий

Средний текст

Очень длинный текст для примера

Ещё

Каждая колонка занимает ровно столько места, сколько нужно её содержимому.

Sass (Syntactically Awesome Style Sheets)

Sass — это препроцессор CSS, который расширяет возможности обычного CSS. Он позволяет использовать переменные, вложенность, миксины и другие возможности, делая написание стилей более удобным и поддерживаемым. Bootstrap написан на Sass, поэтому его легко настраивать под свои нужды.

Основные возможности Sass:

- Переменные** — можно задать значения (цвета, отступы) один раз и использовать их многократно
- Вложенность** — позволяет писать стили в иерархическом виде, как в HTML
- Миксины** — переиспользуемые блоки CSS-кода
- Наследование** — один селектор может наследовать стили другого

Обычный CSS

```
.button {  
  background-color: blue;  
  color: white;  
  padding: 10px;  
}  
  
.button-large {  
  background-color: blue;  
  color: white;  
  padding: 20px;  
}
```

Sass

```
$main-color: blue;  
$text-color: white;  
  
.button {  
  background-color: $main-color;  
  color: $text-color;  
  padding: 10px;  
  
  &-large {  
    padding: 20px;  
  }  
}
```

В примере выше видно, как Sass позволяет избежать повторения кода. Мы создали переменные для цветов и использовали вложенность (`&-large`) для создания модификатора кнопки. Это делает код более чистым и легким в поддержке.

Переменные сетки в Sass

Bootstrap использует следующие переменные для настройки сетки:

```
// Количество колонок
$grid-columns: 12;

// Отступы между колонками (gutter)
$grid-gutter-width: 1.5rem;

// Максимальное количество для .row-cols-*
$grid-row-columns: 6;

// Контрольные точки (breakpoints)
$grid-breakpoints: (
  xs: 0,
  sm: 576px,
  md: 768px,
  lg: 992px,
  xl: 1200px,
  xxl: 1400px
);

// Ширина контейнеров
$container-max-widths: (
  sm: 540px,
  md: 720px,
  lg: 960px,
  xl: 1140px,
  xxl: 1320px
);
```

Миксины для создания своей сетки

```
// Создание контейнера
.my-custom-container {
  @include make-container();
}

// Создание ряда
.my-custom-row {
  @include make-row();
}

// Подготовка колонки
.my-custom-col {
  @include make-col-ready();
  @include make-col(6); // колонка на 6 частей
}

// Колонка со смещением
.my-col-offset-3 {
  @include make-col-offset(3);
}
```

Настройка сетки под себя

Вы можете изменить параметры сетки, переопределив переменные Sass до подключения Bootstrap:

```
// Например, сделать 10-колоночную сетку
$grid-columns: 10;

// Уменьшить отступы между колонками
$grid-gutter-width: 1rem;

// Добавить свою контрольную точку
$grid-breakpoints: (
  xs: 0,
  sm: 576px,
  md: 768px,
  lg: 992px,
  xl: 1200px,
  xxl: 1400px,
  xxxl: 1600px // новая точка
);

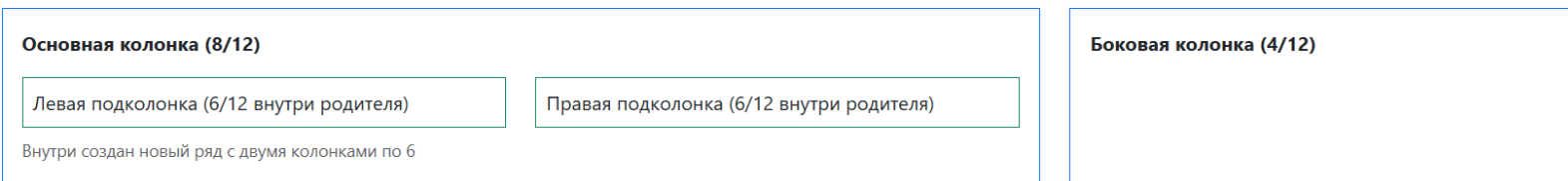
// Затем подключить Bootstrap
@import "bootstrap";
```

Примечание: После изменения переменных нужно перекомпилировать Sass в CSS.

Вложенность сетки

Вложенность — это возможность создавать новую сетку внутри существующей колонки. Это позволяет строить сложные макеты с дополнительной структурой внутри основных блоков. Например, внутри основной колонки с контентом можно создать еще один ряд с несколькими колонками.

Важное правило: Вложенные колонки считаются относительно родительской колонки, а не всей страницы. Если родитель занимает 8 колонок из 12, то внутри него колонка с классом `col-6` будет занимать половину от этих 8 колонок (то есть 4 колонки от общей ширины страницы). Сумма вложенных колонок не должна превышать 12, но может быть меньше.



В данном примере основная колонка занимает 8 частей из 12. Внутри нее создан новый ряд с двумя колонками, каждая из которых занимает по 6 частей внутри родителя. Таким образом, левая и правая подколонки фактически занимают по 4 части от общей ширины страницы (половина от 8).

Что такое формы в Bootstrap?

Формы в Bootstrap — это набор стилей и компонентов для создания удобных, адаптивных и доступных форм. Они включают поля ввода, чекбоксы, радиокнопки, селекты, группы ввода и валидацию. Формы обеспечивают единый вид в разных браузерах и устройствах.

Bootstrap использует классы вроде `form-control` для стилизации элементов, делая их современными и отзывчивыми.

Обзор форм в Bootstrap

Формы строятся на классах вроде `form-control` для полей ввода, `form-label` для меток и `form-text` для подсказок. Например:

```
<form>
  <div class="mb-3">
    <label for="exampleInputEmail1" class="form-label">Адрес электронной почты</label>
    <input type="email" class="form-control" id="exampleInputEmail1" aria-describedby="emailHelp" />
    <div id="emailHelp" class="form-text">Мы никогда не поделимся вашей почтой.</div>
  </div>
  <button type="submit" class="btn btn-primary">Отправить</button>
</form>
```

Адрес электронной почты

Мы никогда не поделимся вашей почтой.

1. Элементы управления формами (Form Controls)

Класс `form-control` применяется к `<input>`, `<textarea>` и `<select>`. Поддерживает типы: text, email, password, number, file и т.д. Размеры: `form-control-sm`, `form-control-lg`.

```
<input type="text" class="form-control" placeholder="Введите текст" />
<textarea class="form-control" rows="3"></textarea>
```

Стандартный текстовый ввод

Маленький численный ввод (.form-control-sm)

Большой парольный ввод (.form-control-lg)

2. Выпадающие списки (Select)

Класс `form-select` для `<select>`. Поддерживает multiple, size, disabled. Размеры: `form-select-sm`, `form-select-lg`.

```
<select class="form-select">
  <option selected>Выберите...</option>
  <option value="1">Один</option>
</select>
```

Выберите...

Отключенный селект

3. Чекбоксы и радиокнопки (Checks and Radios)

Класс `form-check` для чекбоксов и радиокнопок. Для свитчей — `form-switch`. Inline: `form-check-inline`. Без меток: используйте `aria-label`.

```
<div class="form-check">
  <input class="form-check-input" type="checkbox" id="check1" />
  <label class="form-check-label" for="check1">Стандартный чекбокс</label>
</div>
```

☐ Стандартный чекбокс

Варианты расположения и стилизации

Чекбоксы и радиокнопки можно располагать горизонтально, вертикально, делать кнопочного вида и использовать свитчи.

Горизонтальное (Inline) расположение

- ☐ 1 ☐ 2 ☐ 3 (отключено)
- ☐ 1 ☐ 2 ☐ 3 (отключенный)
- ☒ Свитч

Кнопочного вида

Класс `btn-check` + `btn` превращает чекбоксы и радиокнопки в красивые кнопки-переключатели.

Чекбокс 1

Чекбокс 2

Чекбокс 3 (выбран)

Вкл

Выкл

4. Ползунки (Range)

Класс `form-range` для `<input type="range">`. Поддерживает min, max, step, disabled.

<input type="range" class="form-range" min="0" max="5" step="0.5" id="customRange" />

5. Группы ввода (Input Group)

Группы ввода позволяют добавлять текст, иконки, кнопки, выпадающие списки слева и/или справа от поля ввода. Классы: `input-group`, `input-group-text`.

<div class="input-group">
 With textarea
 <textarea class="form-control" aria-label="With textarea"></textarea>
</div>

With textarea

Базовые примеры

@ Username

Recipient's username @example.com

\$.00

С кнопками

Кнопка

Recipient's username Кнопка

Dropdown ▾

С чекбоксами / радиокнопками

☐

☐

Несколько дополнений

С Двумя дополнениями

6. Плавающие метки (Floating Labels)

Класс `form-floating` для плавающих меток над полями. **Требует placeholder.**

```
<div class="form-floating mb-3">
  <input type="email" class="form-control" id="floatingInput" placeholder="">
  <label for="floatingInput">Плавающая метка</label>
</div>
```

Плавающая метка

Плавающие метки с группами ввода

@ Email address

Плавающие метки в сетке

При работе с системой сеток Bootstrap обязательно размещайте элементы формы в классах столбцов.

Email address

Работает с selects
Открыть меню



7. Макеты форм (Layout) — подробный обзор

По умолчанию элементы форм (`input`, `select`, `textarea`) занимают 100% ширины родителя и располагаются вертикально. Для создания многостолбцовых, горизонтальных или адаптивных форм используется система сетки Bootstrap.

Сетка форм

Оборачивайте поля в `.row` и используйте `.col-*`. Горизонтальные отступы регулируются классами `.g-*` (gutters).

```
<div class="bd-example">
  <div class="row g-3">
    <div class="col-md-6">
      <input type="text" class="form-control" placeholder="Имя" />
    </div>
    <div class="col-md-6">
      <input type="text" class="form-control" placeholder="Фамилия" />
    </div>
    <div class="col-12">
      <input type="email" class="form-control" placeholder="Email" />
    </div>
  </div>
</div>
```

Имя

Фамилия

Email

Горизонтальные формы

Используйте класс `.col-form-label` для выравнивания меток по вертикали с полями.

Email

Пароль

Авторазмер колонок (.col-auto)

Колонка занимает ровно столько места, сколько нужно содержимому. Удобно для форм поиска, фильтров, логинов.

Имя пользователя

@ Ник

Отправить

Адаптивные строчные формы (row-cols-auto)

На больших экранах элементы выстраиваются в ряд по горизонтали, на мобильных — стакаются по вертикали.

@ Username

Выберите роль

Войти

8. Валидация форм (Validation)

Bootstrap использует встроенную HTML5-валидацию + стили для состояний `:valid` / `:invalid`. Есть два основных подхода: клиентская (браузерная) и серверная.

Клиентская валидация (рекомендуемый способ)

Самый удобный вариант для пользователя: он ещё не отправил форму, а уже видит «Эй, тут забыли email» или «Пароль слишком короткий».

Что нужно сделать:

1. Добавить на форму `class="needs-validation"` и атрибут `novalidate` (это отключает стандартные всплывающие окошки браузера).
2. На обязательные поля навесить атрибут `required`, тип `type="email"`, `minlength` и т.д. — браузер сам проверит.
3. Под каждым полем добавить блок с сообщением об ошибке: `<div class="invalid-feedback">Текст ошибки</div>`
4. Немного JavaScript, чтобы при отправке формы добавлялся класс `was-validated` — именно он включает красивые стили Bootstrap.

```
<div class="bd-example">
  <form class="needs-validation" novalidate>
    <div class="mb-3">
      <label for="userEmail" class="form-label">Email</label>
      <input type="email" class="form-control" id="userEmail" required />
      <div class="invalid-feedback">Введите нормальный email, пожалуйста</div>
    </div>
    <div class="mb-3">
      <label for="userName" class="form-label">Имя</label>
      <input type="text" class="form-control" id="userName" required minlength="2" />
      <div class="invalid-feedback">Имя должно быть хотя бы из 2 букв</div>
    </div>
    <button class="btn btn-primary" type="submit">Отправить</button>
  </form>
</div>
```

Email

Имя

Отправить

```
<!-- JavaScript (обычно в конце страницы или в отдельном файле) -->
<script>
  (() => {
    'use strict'
    const form = document.querySelector('.needs-validation')
    form.addEventListener('submit', event => {
      if (!form.checkValidity()) {
        event.preventDefault()
        event.stopPropagation()
      }
      form.classList.add('was-validated')
    }, false)
  })()
</script>
```

После этого поле с ошибкой получит красную рамку, под ним появится сообщение, а если всё ок — зелёная рамка и галочка.

Серверная валидация

Если форма уже улетела на сервер, а там выяснилось, что логин занят или код неверный — сервер просто возвращает страницу с подсвеченными полями.

Как сделать:

- К полю добавляем класс `is-invalid` (ошибка) или `is-valid` (всё хорошо).
- Под полем — тот же `invalid-feedback` или `valid-feedback`.
- Если поле внутри `input-group` — добавить класс `has-validation` на группу, иначе рамка может криво отображаться.

```
<div class="bd-example">
  <div class="mb-3">
    <label for="login" class="form-label">Логин</label>
    <input type="text" class="form-control is-invalid" id="login" value="admin" />
    <div class="invalid-feedback">Такой логин уже занят, придумайте другой</div>
  </div>
</div>
```

Логин

admin

Такой логин уже занят, придумайте другой

Где показывать сообщения: под полем или всплывашкой?

- Обычный текст под полем — классы `invalid-feedback` / `valid-feedback`.
- Всплывающая подсказка (`tooltip`) — классы `invalid-tooltip` / `valid-tooltip`. Нужно обернуть поле в `<div class="position-relative">`, подсказка появляется при наведении или фокусе. Используется в случаях, когда места мало.

9. Доступность и отключенные состояния

Bootstrap делает формы максимально доступными, но требует правильного использования атрибутов.

Отключенные поля и группы (disabled / readonly)

Есть три основных состояния:

- disabled** — поле полностью неактивно, не отправляется на сервер, выглядит серым и бледным
- readonly** — поле можно выделять и копировать, но нельзя редактировать, отправляется на сервер
- <fieldset disabled>** — отключает сразу всю группу полей

```
<div class="bd-example">
  <fieldset disabled>
    <legend>Отключенная группа</legend>
    <div class="mb-3">
      <label class="form-label">Имя</label>
      <input type="text" class="form-control" placeholder="Недоступно" />
    </div>
    <div class="form-check">
      <input class="form-check-input" type="checkbox" />
      <label class="form-check-label">Нельзя отметить</label>
    </div>
    <button type="submit" class="btn btn-primary">Отправить</button>
  </fieldset>
</div>
```

Отключенная группа

Имя

Недоступно

☐ Нельзя отметить

Отправить

Доступность: метки и ARIA

ARIA (Accessible Rich Internet Applications) — это набор специальных HTML-атрибутов, которые помогают сделать веб-интерфейсы понятными для скринридеров и других вспомогательных технологий. Они не меняют внешний вид страницы, но сообщают программам чтения с экрана, что именно происходит: где метка, где ошибка, какое поле отключено и т.д.

- Каждое поле ввода должно иметь понятную метку — лучше всего через **<label for="id">**. Если видимой метки нет (например, только иконка), используйте **aria-label** или **aria-labelledby**.
- Подсказки, сообщения об ошибках и пояснения под полем связывайте с помощью **aria-describedby="id-подсказки"**. Тогда скринридеры сразу озвучат важную информацию после поля.
- Для отключённых кнопок и элементов (кроме обычного **disabled**) добавляйте: **aria-disabled="true"** и **tabindex="-1"** — чтобы их не фокусировали клавиатурой.

```
<!-- Пример: поле + подсказка + ошибка -->
<input class="form-control is-invalid" id="email" aria-describedby="emailHelp emailFeedback">
<div id="emailHelp" class="form-text">Мы не передаём почту третьим лицам</div>
<div id="emailFeedback" class="invalid-feedback">Введите корректный email</div>
```

10. Настройка форм через Sass

Bootstrap написан на Sass, поэтому почти все параметры форм (отступы, цвета границ, размер шрифта, толщину фокусной обводки и т.д.) можно переопределить, не трогая основной код.

Самые часто меняемые переменные форм

Переменная	Значение по умолчанию	За что отвечает
<code>\$input-padding-y</code>	.375rem	Вертикальный внутренний отступ поля
<code>\$input-padding-x</code>	.75rem	Горизонтальный внутренний отступ
<code>\$input-font-size</code>	1rem	Размер текста внутри поля
<code>\$input-border-radius</code>	.375rem	Скругление углов
<code>\$input-focus-width</code>	.25rem	Толщина синей обводки при фокусе
<code>\$form-text-color</code>	#6c757d	Цвет мелкого текста-подсказки

Простой пример кастомизации

```
// Перед импортом Bootstrap переопределяем нужные переменные
$input-padding-y: .5rem;           // чуть выше поля
$input-padding-x: 1rem;            // шире поля
$input-font-size: 1.05rem;         // чуть крупнее текст
$input-focus-width: .3rem;         // заметнее фокус
$input-focus-border-color: #6f42c1; // другой цвет обводки при фокусе

// Можно также поменять цвета валидации
$form-valid-color: #198754;
$form-invalid-color: #dc3545;

// Затем подключаем
@import "bootstrap/scss/bootstrap";
```

[Ссылка на все файлы](#)